

Contents

1	Introduction	1
2	Boids	1
2.1	General Procedure, and Parallel Patterns Tehrein	1
3	Implementation	3
3.1	General Schema	3
3.2	Technologies Used	3
3.3	Version Implemented	3
4	Results and Benchmarks	4
4.1	Conclusions	4

1 Introduction

2 Boids

Boids (short for bird-oids, or just birds pronounced with a new york accent) consists of many bird-like (hence the name) objects moving together

- Every boid has a position, and a velocity vector, the immediate behaviour of one boid is determined solely by its current state and by the states of nearby boids
- In the scope of one simulation tick all boid behaviours are computed as if in parallel

2.1 General Procedure, and Parallel Patterns Tehrein

We have implemented a simpler "force field based" approach compared to the original boids paper, this simpler approach has the same parallel characteristics, it just lacks the more physics and control theory based bird simulation of the original paper, this is a shame but the bird simulation didn't introduce any aspects of interest for this exam, so it was skipped.

The procedure is as follows

- For all boids x
 - If it's too close to the area edge or outside of the area, add a force to going back to the area center

- For all other boids y ($O(n^2)$)
 - * If y is too close to x , add a repelling force to x so they don't collide
 - * If y is close enough to x add it as a neighbour of x
- Add a force to x to move its position closer to the average position of its neighbours
- Add a force to x to move its velocity closer to the average velocity of its neighbours

The original boids paper goes over several approaches for combining these forces (there called "potentials", as they act in a more control theory environment), we have opted for a naive weighted sum, followed by clamping of the boid's velocity.

Update computations for one all boids within a tick are independent of each other, as they all depend on the same state, that being the state for the previous tick, and do not modify the state they depend on. It therefore falls under a map parallel programming pattern, and may be considered "embarrassingly parallel".

3 Implementation

3.1 General Schema

Our implementation of the boid algorithms proceeds as follows

- We have two fixed size buffers of boids (number of boids stays fixed throughout simulation)
 - One is to contain the current state of the simulation
 - One is for writing the state of the simulation in the following tick

then at every simulation tick

- For all boids ‘x’ in the first buffer (hic est parallelism)
 - Compute next state of ‘x’
 - Write next state of ‘x’ in second buffer
- Swap buffers for next iteration

3.2 Technologies Used

Program has been developed using C and the OpenMP library/compiler extension, due to use of OpenMP, the sequential and parallel versions of the program can use the same code base, and compilation of the code into one or the other versions may be controlled by turn OpenMP ‘#pragma’s on or off by changing passed compiler flags.

NOTE: this produces compilation warnings for unrecognized pragmas but these warnings are of little concern

Benchmark and graphical versions for the program have been developed, the graphical versions use raylib to handle the graphics

3.3 Version Implemented

Two versions of simulation code have been developed

- one where the data storing the boids, and the simulation loop following it, have data arranged in an AoS (array of structs) manner.
- one where the data storing the boids, and the simulation loop following it, have data arranged in SoA (struct of arrays) manner.

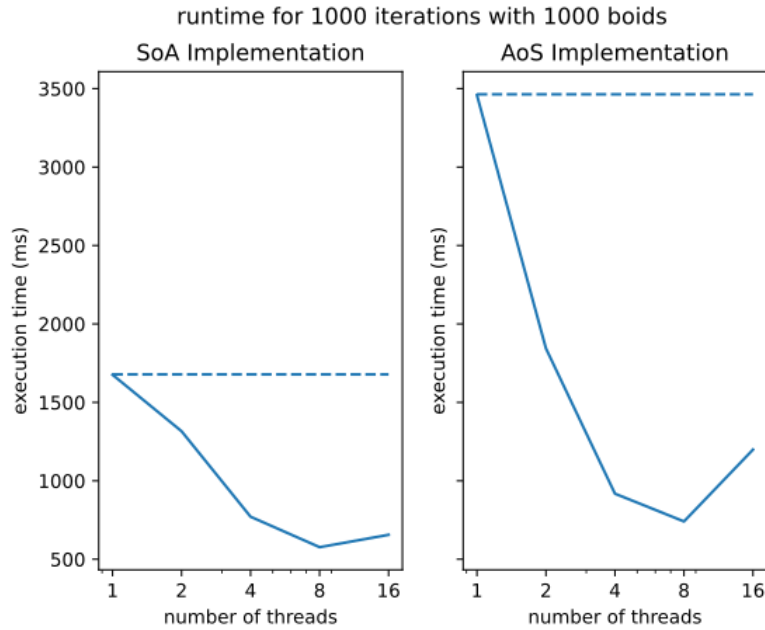
4 Results and Benchmarks

Benchmarks have been run using

- 1000 boids
- 100 warmup iterations
- 1000 timed iterations

Given these parameters, runtime has been measured for different numbers of openmp threads, namely 1, 2, 4, 8, and 16 threads for the parallel versions, sequential versions of the code were also benchmarked with the same amount of boids, warmup iterations, and timed iterations.

Benchmark results are as follows



4.1 Conclusions

Prefacing that the the test machine this has been run on has 8 cores, we may see that

- Adding threads reduces runtime as long as thread number does not exceed number of machine cores, after which increasing threads adds overhead, likely due to excessive context switching.

- Parallel execution with only one thread matches time of sequential execution, meaning synchronization overhead in the 1 thread case may be considered negligible (given our test configuration)
- SoA version of simulation loop scores better than AoS version of the simulation loop for all tried sizes, likely due to better data locality